

EP-06 - Moteur de Planning

Objet : Planification de l'Épique EP-04 - Moteur de Planning (Cœur Critique)

Rôle : Agis en tant que Lead Developer et Product Owner expert en systèmes ERP et algorithmes de planification de ressources.

T'a proposition doit generer des ticket crud pour bien prendre en compte la gestion du planning. En tant que expert en python, po, lead dev ton role est d'indique les ticket à creer, les routes interessant pour bien gerer le planning en prenant en compte les tables qu'il faut. s'achant que le crud sur activité n'est pas encore réalisé. les table consernee son activite, statut planing, planningservice, slot, affectation, membre role, statut affectation, Indiponibilite (crud déjà realiser).

Contexte : > Nous arrivons à la phase finale et critique de l'API "MLA". Les fondations (EP-01), les ressources RH (EP-02) et les contraintes/collectifs (EP-03) sont prêts. Nous devons maintenant coder le **Moteur de Planning (EP-04)** qui orchestre tout.

Données sources (Modèles SQLModel) :

- **Activite** : L'événement physique (Culte, Répétition).
- **PlanningService** : L'instance de planning rattachée à une activité avec un **StatutPlanning**.
- **Affectation** : Le lien avec Role compétence
- Slot: different crenau d'un activité

Livrables attendus :

1. Documentation Confluence (Stratégie de Planning) :

- **Flux de Vie du Planning** : Définis les transitions de **StatutPlanning** (ex: Brouillon → Publié → Clôturé).
- **Logique de Validation Critique** : Explique comment le moteur doit interdire une affectation si :
 - a. Le membre a une **Indisponibilite** sur la date de l'activité.
 - b. Le Chantre est déjà affecté à un autre planning sur le même créneau.
- **Gestion des Rôles** : Comment l'affectation utilise les données de

RoleCompetence

2. **Backlog JIRA (Tickets Techniques)** : Génère des tickets détaillés incluant :

- **Titre & User Story** : Format "En tant que Planificateur, je veux [Action] afin de [Bénéfice]".
- **Volet Qualité & Tests (Pytest)** : Scénarios de "Stress Test" (ex: tester deux affectations simultanées, tester l'affectation d'un choriste à un poste de batteur - doit être possible mais avec avertissement ou bloqué selon la règle).

Contraintes de réalisation :

- **Performance** : Utilisation de `joinedload` pour récupérer l'activité et les affectations en une seule fois.
- **Atomicité** : Utilisation de transactions pour s'assurer qu'une affectation n'est pas créée si la validation échoue.

Format : Tableaux JIRA et sections Confluence claires.

 Analyse du besoin (Pour toi)

L'Épique 04 est **Critique** car elle lie tous tes modèles. Voici ce que tu vas obtenir et pourquoi c'est important :

1. **L'Intégrité Temporelle** : L'IA va devoir définir comment une `Activite` (qui a une date) impacte la `t_affectation`.
2. **Le Workflow de validation** : Contrairement aux épiques précédentes, ici on ne fait pas que du stockage, on fait de la **vérification**.
3. **La vue croisée** : C'est ici que tu pourras sortir des endpoints comme "Donne-moi le planning du Dimanche prochain avec tous les chanteurs et musiciens confirmés".

Priorité	Ticket	Pourquoi ?
----------	--------	------------

P0	Moteur de Collision Temporelle	Sans cela, le planning n'a aucune valeur de vérité.
P0	CRUD Activité & Slots	La structure de base indispensable.
P1	Service d'Affectation avec Validation	Lier les membres aux slots en vérifiant les rôles.
P1	Workflow de Statut (Draft/Published)	Empêcher les membres de voir un planning en cours de modification.
P2	Système de Notifications	Alerter automatiquement par email/push lors de la publication.

Priorité	Titre du Ticket	User Story & Détails Techniques	Critères d'Acceptation (AC)	
P0	[CORE] Moteur de Collision & Chevauchement	<i>En tant que Système, je veux détecter les conflits temporels afin d'empêcher les doubles affectations.</i>	1. Requête SQL optimisée avec index sur les dates. 2. Détection des conflits sur <code>t_indisponib</code>	

		Tech : Créer une fonction utilitaire <code>check_overlap(start, end, member_id)</code> .	<code>ilite</code> ET <code>t_affectation</code> . 3. Retourne une erreur 409 avec le détail du conflit.	
P0	[CRUD] Gestion Activités & Slots	<i>En tant que Planificateur, je veux structurer l'événement et ses créneaux.</i> Tech : CRUD pour <code>t_activite</code> et <code>t_slot</code> .	1. Création cascade : Activité → Planning -> Slots en une fois. 2. Utilisation de <code>joined load</code> pour la lecture. 3. Soft delete géré sur <code>delete_d_at</code> .	
P1	[SERVICE] Affectation avec		<i>En tant que Planificateur, je veux affecter un</i>	1. Vérification de l'existence du rôle

	Validation de Rôles		<i>membre en m'assurant de sa compétence.</i> Tech : Service AssignmentService. Vérification dans t_membre_role .	pour le membre. 2. Blocage ou Warning si rôle manquant (selon paramètre) . 3. Transaction atomique (commit global).
P1	[LOGIC] Workflow de Publication (Statut)	<i>En tant que Responsable, je veux passer le planning de BROUILLON à PUBLIÉ.</i> Tech : State Machine sur PlanningService.statut_code .	1. Les affectations ne sont visibles par le membre que si statut = PUBLIE . 2. Verrouillage des modifications si statut =	

			CLOTUR E .	
P2	[NOTIF] Déclencheur de Notification s de Service	<i>En tant que Membre, je veux être informé dès que mon planning est publié.</i> Tech : Hook post- publication.	1. Détection du passage au statut PUBLIE . 2. Envoi asynchron e (Celery/Ta sk) des notification s aux membres impactés.	

📋 2. Backlog JIRA (Tickets Techniques)

Épique EP-04 : Moteur de Planning

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test (Pytest)
MLA-401	Orchestration Activité & Planning En tant que Planificateur, je veux créer une instance de	• POST /plannin gs : Crée un Planning Service lié à une Activite .	• Tester l'atomicité : Si la création de l'activité échoue, le planning ne doit pas être créé.

	planning pour une activité.	Statut par défaut : BROUILLON.	
MLA-402	Moteur d'Affectation avec Validation En tant que Planificateur, je veux affecter un chantre à un poste spécifique.	- POST /affectations : Reçoit chantre_id, voix_code ou instrument_code. - Retourne 409 Conflict avec détail si indisponible ou déjà pris.	Stress Test : Simuler 2 requêtes simultanées sur le même chantre (Race condition) via des verrous de base de données.
MLA-403	Publication et Verrouillage En tant que Responsable, je veux publier le planning pour informer les équipes.	- PATCH /plannings/{id}/publier. - Vérifie que chaque poste "Principal" est pourvu.	Tester qu'un utilisateur sans rôle "Chef de Chœur" reçoit une 403 Forbidden.

		• Envoi (mock) de notifications.	
MLA-404	Vue Croisée Planning (Read Model) En tant que Chantre, je veux voir mon planning personnalisé.	• GET <code>/me/plannings</code> : Retourne la liste des affectations avec les détails de l'activité. • Utilisation de joinedload pour la performance.	• Vérifier que le JSON de sortie contient bien le nom de l'instrument ou de la voix.

Contraintes de Réalisation & Architecture

Optimisation des Performances (Lead Dev Note)

Pour éviter l'effondrement de la base lors de la consultation des plannings mensuels, nous utiliserons le chargement lié :

1 Python

```

1 # Exemple de requête optimisée pour le ticket MLA-404
2 statement = (
3     select(PlanningService)
4     .options(
5         joinedload(PlanningService.activite),
6         joinedload(PlanningService.affectations).joinedload(Affectation.chantre)
7     )
8 )
9 
```

Atomicité & Transactions

Toute affectation doit être wrappée dans un bloc `try/except` avec un `session.begin_nested()` (savepoint) pour garantir que les validations de rôles techniques (Voix/Instrument) ne corrompent pas l'état du planning en cas d'erreur.

1. 📁 Structure des Répertoires

```
1 Plaintext
2
3 1 app/
4   |__ core/
5   |   |__ database.py           # Configuration DB
6   |__ models/                  # Vos SQLAlchemy Models (t_activite, etc.)
7   |   |__ repositories/        # Couche d'accès aux données (SQL)
8   |       |__ planning_repo.py
9   |       |__ activite_repo.py
10  |__ services/                 # Logique Métier (Validations, Algorithmes)
11     |__ planning_service.py
12     |__ validation_service.py
13  |__ routes/                   # API Endpoints (FastAPI)
14     |__ planning_router.py
```

2. 📝 Couche Repository (`repositories/`)

Le repository ne contient que des requêtes SQL (via SQLAlchemy). Il ne prend aucune décision métier.

`repositories/planning_repo.py`

Fonction	Description	Requête SQL / SQLAlchemy
<code>get_planning_full</code>	Récupère le planning avec toute l'arborescence (optimisé N+1).	<code>select(PlanningService).options(selectinload(Slot.affectations)...))</code>
<code>get_membre_affectations</code>	Récupère toutes les affectations d'un membre sur une période.	<code>select(Affectation).where(Affectation.membre_id == id & date_overlap)</code>

<code>create_assignment</code>	Enregistre une affectation dans la base.	<code>session.add(affectation)</code>
--------------------------------	--	---------------------------------------

`repositories/activite_repo.py`

Fonction	Description	Requête SQL / SQLAlchemy
<code>create_activite</code>	CRUD Activité.	<code>session.add(activite)</code>
<code>get_activite</code>	Récupère une activité.	<code>session.get(Activite, id)</code>

3. 🧠 Couche Service (`services/`)

C'est ici que réside **l'intelligence du Moteur**.

`services/validation_service.py` (Moteur de Règles)

Fonction	Description	Logique
<code>check_member_availability</code>	Vérifie <code>t_indisponibilite</code> .	Retourne <code>False</code> si overlap.
<code>check_member_role</code>	Vérifie <code>t_membre_role</code> .	Retourne <code>False</code> si compétence manquante.
<code>check_double_booking</code>	Vérifie <code>t_affectation</code> .	Analyse les slots surchargés.

`services/planning_service.py` (Orchestrateur)

Fonction	Description	Logique
----------	-------------	---------

<code>assign_member</code>	Fonction Critique. Ordonne les validations et commit.	<code>begin_transaction()</code> → Validations → <code>commit()</code> ou <code>rollback()</code> .
<code>publish_planning</code>	Change le statut et déclenche les notifications.	Update <code>StatutPlanning</code> + Async Task.

4. 🌐 Couche Routes (`routes/`)

Le router valide les entrées (Pydantic) et appelle le service.

`routes/planning_router.py`

Route HTTP	Fonction	Description
POST <code>/activities</code>	<code>create_activity</code>	Crée la structure.
GET <code>/planning/{id}</code>	<code>get_planning</code>	Affiche le planning complet.
POST <code>/slots/{id}/assign</code>	<code>assign_member</code>	Tente une affectation (appel service).
PATCH <code>/planning/{id}</code>	<code>update_planning_status</code>	Publie ou clôture.

💡 Exemple de code complet pour le flux d'affectation

Voici comment le service orchestre la logique pour garantir la cohérence :

```

1 # services/planning_service.py
2
3 class PlanningService:
4     @staticmethod
5     def assign_member(session: Session, slot_id: str, member_id: str,
6 role_code: str):
7         # 1. Start Transaction
8         with session.begin():
9             # 2. Récupérer les données avec eager loading
10             slot = PlanningRepo.get_slot_with_details(session,
11 slot_id)
12
13             # 3. VALIDER RÈGLES MÉTIER
14             # Check indisponibilité
15             if not
16 ValidationService.check_member_availability(session, member_id,
17 slot.debut, slot.fin):
18                 raise ConflictException("Le membre est indisponible")
19
20             # Check double affectation
21             if not ValidationService.check_double_booking(session,
22 member_id, slot.debut, slot.fin):
23                 raise ConflictException("Le membre est déjà affecté
24 ailleurs")
25
26             # Check compétence
27             if not ValidationService.check_member_role(session,
28 member_id, role_code):
29                 # On peut choisir de bloquer ou de lever un
30 avertissement
31                 raise ValidationException("Compétence non valide")
32
33             # 4. CRÉER AFFECTATION
34             new_assign = Affectation(slot_id=slot_id,
35 membre_id=member_id, role_code=role_code)
36             PlanningRepo.create_assignment(session, new_assign)
37
38             return new_assign
39
40

```